

Algorytm A*

Inżynieria Wiedzy i Symboliczne Uczenie Maszynowe

Karol Capała

2026

1 Wprowadzenie do A*

W jaki sposób znaleźć najkrótszą ścieżkę między dwoma wybranymi punktami w grafie o nieujemnych wagach krawędzi? Po studiach pierwszego stopnia naturalną, nasuwającą się odpowiedzią jest algorytm Dijkstry. Dla każdego wierzchołka algorytm wyznacza jego aktualnie najkrótszą znaną odległość od wierzchołka początkowego. Oznaczmy tę odległość jako

$$g(x_i), \quad (1)$$

gdzie x_i jest wierzchołkiem grafu. Następnie algorytm przetwarza wierzchołki w kolejności rosnącej wartości $g(x_i)$, to znaczy w pierwszej kolejności wybiera ten wierzchołek, który jest najbliżej źródła spośród jeszcze nieprzetworzonych. Ponieważ wszystkie wagi krawędzi są nieujemne, w momencie wyboru wierzchołka x_i mamy gwarancję, że wyznaczona wartość $g(x_i)$ jest ostateczną, najkrótszą możliwą odległością od startu do x_i . W szczególności oznacza to, że gdy algorytm po raz pierwszy wybierze wierzchołek docelowy, jego wartość $g(x_{cel})$ jest długością najkrótszej ścieżki między początkiem a końcem.

Takie podejście może być jednak nieefektywne, gdyż aby uzyskać tę gwarancję, algorytm często musi wyznaczyć poprawne odległości do wielu wierzchołków, które w rzeczywistości nie leżą na najkrótszej ścieżce do celu, co w praktyce może oznaczać przeszukiwanie znacznej części grafu, zanim zostanie osiągnięty wierzchołek docelowy.

Celem algorytmu A* (A-gwiazdka, A-star) jest ograniczenie liczby wierzchołków przeszukiwanych w celu znalezienia najkrótszej ścieżki. Osiąga to poprzez wprowadzenie dodatkowej funkcji $h(x_i)$, nazywanej heurystyką, która szacuje koszt (odległość) od wierzchołka x_i do wierzchołka docelowego. Algorytm A* działa więc podobnie do algorytmu Dijkstry, jednak zamiast minimalizować jedynie odległość od źródła, minimalizuje funkcję kosztu całkowitego, będącą sumą rzeczywistej odległości od startu i oszacowania heurystycznego do celu:

$$f(x_i) = g(x_i) + h(x_i). \quad (2)$$

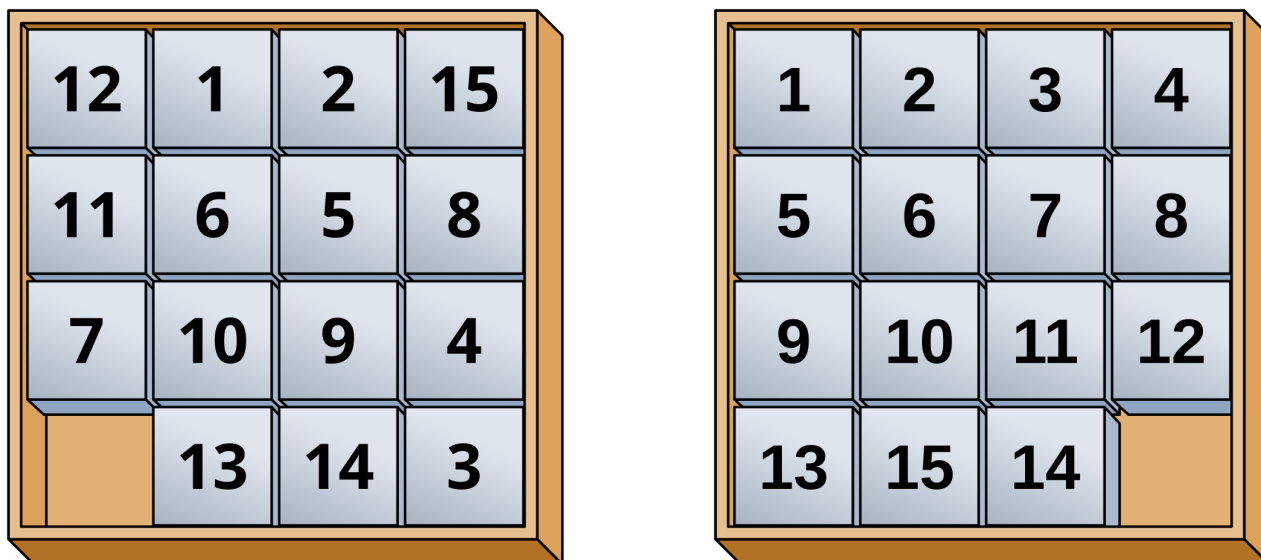
Dobór odpowiedniej heurystyki, jak można się domyślić, zależy silnie od rozważanego problemu i ma kluczowy wpływ na efektywność algorytmu. Algorytm A* stosuje się między innymi do wyznaczania tras między punktami na mapie oraz do przeszukiwania drzew stanów w grach. My skupimy się na tym drugim zastosowaniu na przykładzie gry nazywanej po angielsku N-Puzzle, a po polsku "piętnastką", chociaż w naszym przypadku zaczniemy od uproszczonej wersji tej łamigłówki.

Serdecznie zachęcam również do zapoznania się z filmikiem w języku angielskim z kanału Polylog na platformie Youtube, link jest też załączony do zadania na platformie UPeL

2 N-Puzzle (Piętnastka)

Wyobraźmy sobie obrazek podzielony na równe, kwadratowe fragmenty, które następnie zostały losowo przemieszane. Część z Państwa mogła spotkać się z taką formą łamigłówki, w której celem jest odtworzenie oryginalnego obrazu poprzez przesuwanie fragmentów na właściwe miejsca. Kluczową cechą tego typu układanki jest obecność jednego pustego miejsca, które umożliwia przemieszczanie sąsiednich elementów. Każdy ruch polega na przesunięciu jednego fragmentu obrazu na puste pole, co powoduje zmianę aktualnego ułożenia elementów. Problem ten można uogólnić, rozważając planszę o wymiarach $n \times n$, zawierającą $N = n^2 - 1$ ponumerowanych elementów oraz jedno pole puste. Tak zdefiniowaną łamigłówkę nazywamy problemem N-Puzzle. Stan układanki można interpretować jako pewną konfigurację elementów na planszy, natomiast celem jest osiągnięcie konfiguracji uporządkowanej według ustalonego schematu.

Uwaga! Nie każda konfiguracja początkowa jest rozwiązywalna. Do testów najlepiej wykorzystywać sprawdzone konfiguracje np. z wybranego przez siebie generатора stanów startowych dla N-Puzzle (porównanie metod, podobne z generatorem) lub stanów używanych w linkach załączonych do zadania. Można też napisać własny generator.



Rysunek 1: Przykładowy stan początkowy łamigłówki N Puzzle (po lewej) i stan końcowy (docelowy). Źródło: Wikipedia: 15 puzzle

3 Implementacja A* do N-Puzzle

Poniższa część została przygotowana na podstawie artykułu.

Implementacja algorytmu A* dla problemu N-Puzzle wymaga stworzenia drzewa możliwych ruchów. Zaczynamy od określenia, jakie informacje muszą być przechowywane w węźle. Musimy przechowywać:

- stan układanki w danym momencie, czyli tablicę o wymiarach $N \times N$,
- wartość funkcji heurystycznej h ,
- wskaźnik do węzła nadrzędnego.

Możemy również przechowywać wiersz i kolumnę pustego pola, aby przyspieszyć działanie algorytmu, jednak nie jest to wymagane. W związku z tym struktura węzła może mieć następującą postać:

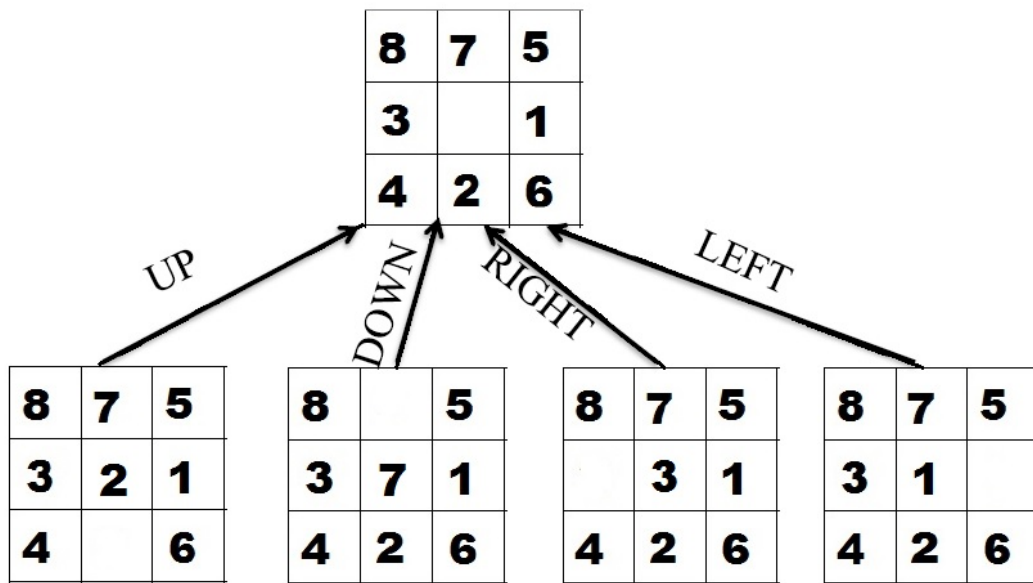
```
Structure Node
{
    variable array grid[N][N], variable h, variable g;
    pointer parent;
}
```

3.1 Tworzenie grafu

Jak omówiono wcześniej, przestrzeń przeszukiwania należy interpretować jako graf. Każdy stan w grafie jest reprezentowany przez konfigurację układanki, a zatem każdy węzeł odpowiada jednemu stanowi układanki, który powstaje poprzez przesunięcie kafelka na puste pole ze stanu poprzedniego.

8	7	5
3		1
4	2	6

Przedstawiona konfiguracja stanowi pojedynczy węzeł grafu, który możemy traktować jako węzeł początkowy. W kolejnym kroku graf rozszerza się poprzez wykonanie wszystkich możliwych ruchów.



Każdy węzeł może mieć maksymalnie czworo potomków. Graf rozwija się dalej w analogiczny sposób, przy czym każdy węzeł potomny zawiera wskaźnik do swojego węzła nadrzędnego.

Wartość funkcji kosztu g , która reprezentuje koszt wykonanych ruchów, zwiększa się o 1 na każdym poziomie głębokości. Wynika to z faktu, że każde przesunięcie kafelka na puste pole odpowiada jednemu ruchowi, a koszt takiego ruchu jest w tym zadaniu stały.

3.2 Heurystyki

Istnieje kilka heurystyk, które można zastosować w problemie N-Puzzle.

3.2.1 Odległość Hamminga / liczba błędnie położonych kafelków

Heurystyka ta zwraca liczbę kafelków, które nie znajdują się na swoich docelowych pozycjach. Rozważmy przykładową konfigurację układanki.

8	4	5
3		1
7	2	6

W tej konfiguracji tylko jeden kafelek znajduje się na poprawnej pozycji "7", natomiast pozostałe są niepoprawnie rozmieszczone. Wartość heurystyki h wynosi więc liczbę wszystkich niepoprawnie położonych kafelków. Podczas obliczania tej heurystyki nie uwzględnia się pustego pola. Heurystyka ta jest najmniej efektywna, ponieważ wymaga przeszukania dużej liczby stanów przed osiągnięciem rozwiązania.

3.2.2 Odległość Manhattan

Odległość Manhattan dla pojedynczego kafelka jest równa liczbie przesunięć wymaganych do osiągnięcia jego pozycji docelowej. Dla całego stanu układanki heurystyka ta jest sumą odległości Manhattan wszystkich kafelków, z pominięciem pustego pola.

1	2	5
3		6
7	4	8

Rozważmy przykładową konfigurację, w której kilka kafelków znajduje się poza swoimi pozycjami docelowymi. Dla każdego z tych kafelków obliczamy jego odległość Manhattan. Wartość heurystyki h jest sumą tych odległości. Heurystyka oparta na odległości Manhattan jest bardziej efektywna niż heurystyka Hamminga, jednak dla większych wartości N nadal może być niewystarczająco szybka.

3.2.3 Konflikt liniowy + odległość Manhattan

Dwa kafelki znajdują się w konflikcie liniowym, jeżeli są w tym samym wierszu lub kolumnie, ich pozycje docelowe znajdują się również w tym samym wierszu lub kolumnie, a jeden kafelek blokuje drugiemu dostęp do jego pozycji docelowej.

4	2	5
1		6
3	8	7

W takiej sytuacji rozwiązanie konfliktu wymaga wykonania dodatkowych ruchów. Każdy konflikt liniowy zwiększa wartość heurystyki o 2, ponieważ jeden kafelek musi najpierw ustąpić miejsca drugiemu, a następnie powrócić na właściwą pozycję.

Heurystyka konfliktu liniowego jest zawsze stosowana łącznie z odległością Manhattan. Wartość heurystyki oblicza się jako:

$$h = \text{odległość Manhattan} + 2 \times \text{liczba konfliktów liniowych}. \quad (3)$$

Heurystyka ta jest znacznie bardziej efektywna niż wcześniej opisane heurystyki. Pozwala ona rozwiązywać układanki o rozmiarze 4×4 w rozsądnym czasie. Podobnie jak w innych heurystykach, pustego pola nie uwzględnia się podczas obliczeń.

3.2.4 Bazy heurystyk

Istnieją również bazy podające heurystyki dla poszczególnych sytuacji na planszy, jednak wykraczają one poza zakres tego zadania.

3.3 Pseudokod

Algorithm 1 A* dla N-Puzzle z heurystyką Manhattan + Linear Conflict

```
1: function ASTAR(start, goal)
2:   open  $\leftarrow$  priority queue ordered by  $f(n) = g(n) + h(n)$ 
3:   closed  $\leftarrow \emptyset$ 
4:    $g(\textit{start}) \leftarrow 0$ 
5:    $h(\textit{start}) \leftarrow \text{HEURISTIC}(\textit{start}, \textit{goal})$ 
6:    $f(\textit{start}) \leftarrow g(\textit{start}) + h(\textit{start})$ 
7:   insert start into open
8:   while open is not empty do
9:     current  $\leftarrow$  node from open with lowest f
10:    if current = goal then return RECONSTRUCTPATH(current)
11:    end if
12:    remove current from open
13:    add current to closed
14:    for all neighbor in GENERATENEIGHBORS(current) do
15:      if neighbor  $\in$  closed then
16:        continue
17:      end if
18:      tentative_g  $\leftarrow g(\textit{current}) + 1$ 
19:      if neighbor  $\notin$  open OR tentative_g <  $g(\textit{neighbor})$  then
20:        parent(neighbor)  $\leftarrow$  current
21:         $g(\textit{neighbor}) \leftarrow \textit{tentative\_g}$ 
22:         $h(\textit{neighbor}) \leftarrow \text{HEURISTIC}(\textit{neighbor}, \textit{goal})$ 
23:         $f(\textit{neighbor}) \leftarrow g(\textit{neighbor}) + h(\textit{neighbor})$ 
24:        if neighbor  $\notin$  open then
25:          insert neighbor into open
26:        end if
27:      end if
28:    end for
29:  end while
30:  return failure
end function
```

Algorithm 2 Heurystyka Manhattan + Linear Conflict

```
1: function HEURISTIC(state, goal)
2:   manhattan  $\leftarrow$  0
3:   linear_conflict  $\leftarrow$  0
4:   for all tile t  $\neq$  blank do
5:     (r, c)  $\leftarrow$  current position of t
6:     (rg, cg)  $\leftarrow$  goal position of t
7:     manhattan  $\leftarrow$  manhattan +  $|r - r_g| + |c - c_g|$ 
8:   end for
9:   for all each row r do ▷ Sprawdź konflikty w wierszach
10:    for all each pair of tiles (ti, tj) in row r do
11:      if ti and tj belong in row r in goal AND goal_col(ti) > goal_col(tj) then
12:        linear_conflict  $\leftarrow$  linear_conflict + 2
13:      end if
14:    end for
15:  end for
16:  for all each column c do ▷ Sprawdź konflikty w kolumnach
17:    for all each pair of tiles (ti, tj) in column c do
18:      if ti and tj belong in column c in goal AND goal_row(ti) > goal_row(tj) then
19:        linear_conflict  $\leftarrow$  linear_conflict + 2
20:      end if
21:    end for
22:  end for
23:  return manhattan + linear_conflict
24: end function
```

Algorithm 3 Generowanie sąsiadów

```
1: function GENERATENEIGHBORS(state)
2:   neighbors  $\leftarrow$   $\emptyset$ 
3:   (r, c)  $\leftarrow$  position of blank tile
4:   for all (dr, dc)  $\in \{(-1, 0), (1, 0), (0, -1), (0, 1)\}$  do
5:     (r', c')  $\leftarrow$  (r + dr, c + dc)
6:     if (r', c') inside board then
7:       new_state  $\leftarrow$  copy of state
8:       swap blank with tile at (r', c')
9:       add new_state to neighbors
10:    end if
11:  end for
12:  return neighbors
13: end function
```

4 Zadanie

W oparciu o przedstawiony opis i pseudokod zaimplementuj rozwiązanie łamigłówki N-Puzzle, dla $N = 8$ wykorzystując 3 wybrane heurystyki i porównaj koszt działania.

Zadanie dodatkowe Rozszerz kod na przypadek $N = 15$, porównaj A* z prostą heurystyką do takiego z bazą wzorców (Pattern Database) lub algorytmu IDA*.

Uwaga! Unikaj testowania bez zaimplementowanej heurystyki, jako że drzewo możliwych ruchów jest ogromne i jego przeszukanie będzie bardzo czasochłonne.